



DSA Notes (Cleaned, Corrected, & Structured for Revision)



How to Think Where This Question Should Go

- If **brute force is linear ($O(n)$)** and giving **TLE**, you often need to bring it down to **$O(\log n)$** or **constant time**.
 - When you think of `log(n)` time: 🔥 Think **Binary Search** or **Priority Queue**.
-



Two-Pointer Technique

1. Opposite Direction Pointers

Used when comparing/extremes from both ends (e.g., sorted arrays).

Common Use Cases:

- Palindrome check
- Container With Most Water
- Sorted squares
- Trapping Rain Water

Template:

```
int left = 0;
int right = arr.length - 1;
while (left < right) {
    // process
    if (condition) left++;
    else right--;
}
```

Intuition:

- Squeeze from both ends
- Good when one side complements the other (e.g. max-min)
- Condition to move left/right is often based on which value is greater/smaller

Problems:

- Two Sum II
 - Valid Palindrome
 - Squares of Sorted Array
 - Trapping Rain Water
 - Container With Most Water
 - Reverse String
-

2. → Same Direction (Sliding Window)

Use when shrinking/expanding subarrays/substrings to satisfy a condition.

Common Use Cases:

- Longest substring without repeating characters
- Minimum size subarray sum
- Max consecutive 1s with at most k flips

Template:

```
int left = 0;
for (int right = 0; right < n; right++) {
    // include s[right] in window
    while (window violates condition) {
        // exclude s[left] from window
        left++;
    }
    // update result
}
```

Problems:

- Longest Substring Without Repeating Characters
 - Min Size Subarray Sum
 - Max Consecutive Ones III
 - Longest Repeating Character Replacement
 - Fruit Into Baskets
-

Prefix Sum + HashMap (Subarray Sums)

- Used in counting subarrays with specific sum properties
- Key concept in problems like:

- 560. Subarray Sum Equals K -
 1. Binary Subarrays With Sum -
 2. Subarray Sums Divisible by K

Intuition:

- Prefix sum helps track running total
- Store count of prefix sums in HashMap
- If (currSum - target) exists in map => found a valid subarray

When to Use Map and Set

- If you want to know: *"Did this element appear before?"* → Think Set
- If you want to associate data with an element → Think Map
- Need uniqueness? Use Set
- Need lookup + data? Use Map

Problem Intuition:

- "If I could instantly know if this element or prefix existed earlier..." → Use map/set

Rotations, Reversals & Reordering

Left Rotate Array by k

- Use reverse technique:
- Reverse first k elements
- Reverse last n-k elements
- Reverse whole array

Intuition:

- Problems involving shifting/rotating/rearranging in-place
- Reversal helps avoid extra space

Merge Sort Patterns

When array must stay in order, but sorting helps — think merge sort.

Use when:

- Counting inversions: $i < j \ \&\& \ arr[i] > arr[j]$
- Reverse pairs

Problems:

- Count Inversions
 - Reverse Pairs
-

Binary Search Variants

A. Standard Binary Search (Sorted Arrays)

- Move left/right based on condition
- Understand condition changes at `mid`

Tip:

- Use answer variable when finding best candidate (e.g., lower bound, upper bound)

B. Rotated Binary Search

- One side is always sorted
- Decide which side to move based on sorted half

C. Binary Search on Answer

Use when searching over a range of possible answers.

Common Setup:

- low = min possible answer
- high = max possible answer

Problems:

- Minimum Max Value Problems
- Square Root
- Koko Eating Bananas
- Split Array Largest Sum (Leetcode 410)

D. Max(Min) or Min(Max)

Used in optimization problems:

- Minimize max workload / sum / value
 - Maximize min distance / profit / value
-

Grouping / Sorting with Unique Values

- When few unique values: Use grouping/counting instead of sorting
- Avoid sorting whole array if grouping is enough

Problem Examples:

- Sort Colors (0s, 1s, 2s) – Dutch National Flag
 - Group Anagrams
-

9 Brute Force Validity

When number of **moves or constraints are very small** (e.g., ≤ 3 changes):

- Try brute force (all possibilities)

Example:

- Minimum Difference Between Largest and Smallest Value in Three Moves
-

Matrix Problem Strategy

- Always map position from original to new
- Think:
 - What was the original coordinate?
 - Where does it move in the transformation?
- Use mapping for:
 - Rotations
 - Reflections
 - Spirals

Strategy:

- Try to find relation between old and new positions
-

General Problem Strategy

“If brute force is TLE, and you want $\log(n)$: Binary Search or Priority Queue”

“If sorted array + max/min constraints: Binary Search on Answer”

“If checking previous elements or building history: Prefix Sum + Map”

"If moving forward with condition: Two Pointer (Sliding Window)"

"If squeezing from both ends: Two Pointer (Opposite Direction)"

"If few distinct elements or swaps allowed: Try Brute Force"

"If you wish array was sorted but can't sort it: Merge Sort-based Trick"

"If you need uniqueness + speed: Set / Map"

"If there's a pattern in matrix position change: Map old position to new"



Practice Pattern Tracker

-

Want me to help build short flashcards or problem ladders for each of these patterns?